CSE 598: Data-Intensive Systems for Machine Learning - Project Proposal

Title and Group Members

Project Title: Access-Aware KV-Cache Sharing for Resource-Constrained RAG Serving

Group Members: Vedang Vasant Avaghade (vavaghad@asu.edu), Sri Harshith Goli (sgoli125@asu.edu)
Group Leader: Equal Contributions from both.

Background and Motivation

Application/system studied. We study multi-user Retrieval-Augmented Generation (RAG) serving. A RAG assistant retrieves documents and places them in an LLM prompt. In an organization, those documents may have different access rules: a handbook may be public to employees, a procedure may be limited to one group, and a personal record may be visible to one user. Reprocessing the same long context for many users wastes GPU work and scarce KV-cache memory.

How existing systems address the problem. LLM serving engines such as vLLM use a *KV cache*: once a prompt prefix is processed, its intermediate computation can be reused when later requests begin with the same tokens [1]. To reduce cross-user timing leakage, vLLM supports a request-level `cache_salt`; different salts isolate the request's cache-hash chain [2].

Limitations of existing approaches. Current request-level isolation is coarse. A global cache can provide high reuse but may reveal cross-user cache state through timing, while a per-user salt is safer but prevents two authorized users from sharing computation even for identical public or group documents. Thus, **the cache treats the entire prompt as one trust scope even when the prompt contains several access scopes**. Under a small KV-cache budget, this can duplicate identical content, cause earlier eviction, and repeat expensive prefill computation. A newly disclosed vLLM advisory also shows why cache-scope metadata must be handled carefully: versions before 0.29.0 accepted an unbounded `cache_salt`, which allowed very large salts to consume the single EngineCore scheduler thread and stall unrelated requests, vLLM 0.29.0 patches this issue [9]. Our design therefore keeps scope identifiers server-derived, fixed-size, and outside direct user control.

Why solving this is useful. Enterprise RAG already knows who may read each document. If this ACL information can safely guide cache sharing, the serving system could reduce repeated GPU computation and improve latency/throughput without weakening access control. The goal is not to share more data, but to avoid recomputing work that multiple users are already authorized to reuse.

Related work. CachePrune studies fine-grained privacy-aware KV reuse [4], KVGov studies isolation against KV-cache timing leakage [5], and CacheWeaver reorders RAG evidence to create reusable prefixes [6]. PrefixWall dynamically monitors cross-user prefix reuse, flags suspicious sharing, and selectively isolates prefixes to mitigate timing side channels [8]. Our approach is complementary: instead of deciding isolation from observed behavior, it uses authoritative document ACLs *before reuse* to determine who is allowed to share each prefix. A 2025 vLLM RFC proposed multiple salt barriers within a request, although current documented vLLM behavior uses a single request-level salt [3, 2]. Our project therefore does not claim that "multiple salts" alone are new. We investigate whether **document ACLs can jointly guide safe cache scopes and RAG context construction** under limited KV-cache memory.

Research Problem Statement

Problem. Given multi-user RAG requests whose retrieved documents carry authoritative ACL metadata, how can an LLM serving system maximize *authorized* cross-user KV-cache reuse under a fixed GPU KV-cache budget, while preventing unauthorized cache reuse and preserving answer quality?

1) Input/workload. Each request contains an authenticated user, group memberships, a query, and retrieved chunks with ACLs such as `public`, `group:hr`, or `user:alice`. We assume retrieval has already removed documents the requester cannot read.

2) Behavior to improve. We want the serving system to reuse more already-computed prompt/KV state across users who are jointly authorized, instead of recomputing identical RAG prefixes.

3) Important constraints. GPU KV-cache memory is limited. Security is a hard constraint: a requester must never obtain a cache hit for a prefix whose effective ACL does not authorize that requester. RAG answer quality should remain comparable to normal retrieval ordering.

4) Objectives. Minimize recomputed prompt tokens and TTFT, while improving cache-hit rate and throughput under the fixed KV-cache budget. Security violations must remain zero and answer quality should not materially degrade.

Research questions.

1. **RQ1:** Can ACL-derived cache scopes reduce prompt recomputation and TTFT versus safe per-user isola-

tion without unauthorized cache hits?
2. **RQ2:** Can access-aware evidence ordering further increase safe cache reuse without materially reducing QA quality?
3. **RQ3:** Under what sharing rates, concurrency levels, and KV-cache sizes does ACL-aware sharing provide meaningful benefit?

Technical Challenges

The following challenges make the problem nontrivial. For each one, we state why it occurs, why a straightforward solution is insufficient, and how existing approaches leave room for our study.

C1 - Prefix state inherits earlier restrictions.

- **Why it occurs:** KV state for a token depends on all earlier tokens. A public document placed after private evidence therefore has a private-influenced prefix.
- **Why the simple solution is insufficient:** Labeling each block only with the current document's ACL can incorrectly mark such state as public.
- **Existing approaches / gap:** Request-level salting protects the whole request, but is too coarse. We need the effective ACL of the complete prefix.

C2 - Security and reuse pull in opposite directions.

- **Why it occurs:** More sharing saves GPU work, but sharing across the wrong users can expose restricted cache state.
- **Why the simple solution is insufficient:** Per-user isolation is safe but loses legitimate public/group reuse; global sharing is efficient but unsafe.
- **Existing approaches / gap:** vLLM supports request-level isolation, and prior work studies cache privacy, but our workload needs ACL-derived sharing boundaries inside RAG prompts.

C3 - Better cache order may conflict with RAG order.

- **Why it occurs:** Broadly shareable evidence placed first creates longer reusable prefixes, while retrieval rank normally reflects relevance.
- **Why the simple solution is insufficient:** Always sorting by ACL may move important evidence and reduce answer quality.
- **Existing approaches / gap:** CacheWeaver studies cache-aware evidence ordering, but not ordering constrained by document ACLs and secure cache scopes.

C4 - Benefits depend on workload.

- **Why it occurs:** Reuse is valuable only when authorized users repeatedly retrieve overlapping evidence under cache pressure.
- **Why the simple solution is insufficient:** A single favorable workload could overstate the benefit.
- **Existing approaches / gap:** We therefore vary overlap, concurrency, ACL mix, and cache size to find the break-even region rather than assume universal gains.

Initial Technical Ideas

We keep one research problem and evaluate increasingly capable ideas. Each idea is tied to the challenges above.

Idea A - ACL-derived multi-scope caching.

- **Basic mechanism:** The server derives an effective authorization set for each prefix, $\mathrm{ACL}(P_k) = \bigcap_{i=1}^{k} \mathrm{ACL}(D_i)$. When it changes, a server-generated keyed scope identifier is added to the cache hash; later blocks inherit that restriction.
- **Challenge addressed:** C1 and C2 – prefix-dependent authorization and the security/reuse trade-off.
- **Difference from baseline/existing work:** Unlike one per-user salt or one shared/private boundary, cache scopes are derived from the ACLs of the RAG evidence and may change multiple times within one request. We treat multi-barrier hashing as a mechanism from prior work, not as novelty by itself.
- **Why we expect it to work:** Users with the same effective ACL can hash identical authorized prefixes to the same cache blocks, while different ACLs produce separated namespaces.

Idea B - Access-aware evidence ordering.

- **Basic mechanism:** After authorization filtering, place evidence with broader compatible ACLs earlier while preserving retrieval rank as much as possible within each group.
- **Challenge addressed:** C3 – increasing reusable prefix length without sacrificing RAG quality.
- **Difference from baseline/existing work:** Normal RAG follows retrieval order, our ordering explicitly uses ACL structure together with relevance. We compare both orders using the same cache mechanism.
- **Why we expect it to work:** A prefix remains reusable by a larger audience until narrower evidence is introduced, so broad-to-narrow ordering should delay the point at which sharing becomes restricted.

Idea C - Reuse-aware ordering (stretch).

- **Basic mechanism:** Use a simple heuristic score that combines retrieval relevance with estimated authorized reuse (for example, favoring broadly authorized, frequently repeated chunks). This is intentionally a lightweight stretch goal, we will not build a complex offline oracle.

- **Challenge addressed:** C3 and C4 – balancing answer quality with workload-dependent cache benefit.
- **Difference from baseline/existing work:** Rather than using a fixed broad-to-narrow rule, the policy adapts to estimated reuse and cache pressure.
- **Why we expect it to work:** When overlap is high, frequently reusable evidence should be worth moving earlier; when overlap is low, the score can stay close to normal retrieval order.

Evaluation Plan

Platform. We will use ASU Research Computing (or an equivalent available server) with one NVIDIA A100 GPU and vLLM. We will limit KV-cache capacity to 1, 2, and 4 GiB using vLLM's cache configuration so that memory pressure is reproducible [7].

Models. We will use a 7–8B instruction model, primarily Qwen2.5-7B-Instruct or Qwen3-8B. These models are large enough to exercise realistic LLM serving behavior but small enough to fit on one A100. The project requires inference only; no model training, CUDA kernels, or architecture changes are needed.

Datasets and workloads. We will use HotpotQA or Natural Questions as the document/question source and assign controlled synthetic ACLs to chunks (public, group, user-private). For each experimental ACL mix, chunks will be randomly assigned to the public, group, or user-private class according to fixed target proportions; group chunks will then be assigned uniformly across the simulated groups and private chunks uniformly across users, using fixed random seeds for reproducibility. We will simulate 20–100 users and 4–10 groups while varying shared-document rate (0, 25, 50, 75%), concurrency, ACL mix, and KV-cache capacity. Synthetic ACLs provide exact ground truth for whether each cache hit is authorized.

Resource feasibility. A 7–8B model fits on an A100 for inference, and our main experimental resource is the intentionally restricted KV cache rather than total GPU capacity. We will begin with a small 20-user workload and scale upward. If A100 time is limited, the same experiments can use a smaller instruction model while preserving the cache policy and workload methodology.

Baselines/variants. (B0) no prefix caching; (B1) safe per-user request salt; (B2) one shared/private boundary; (B3) ACL-derived multi-scope caching with normal retrieval order; (B4) B3 plus access-aware ordering. Unrestricted global sharing will be reported only as an *efficiency upper bound*, not as a secure baseline.

Metrics. Performance: p50/p95 TTFT, requests/s, cached-token percentage, recomputed prompt tokens, unique KV blocks, and evictions. Security: number of unauthorized restricted-block hits (required: zero) plus a timing-probe experiment comparing insecure global sharing with our scoped design. Quality: Exact Match/F1 or the benchmark's standard QA metric before and after evidence reordering, plus a semantic metric (e.g., embedding-based answer similarity) or a fixed-rubric LLM-as-a-judge check so correct paraphrases are not falsely counted as failures.

Decision criteria. We will not assume our approach always wins. We will report where it helps and where it does not. A successful result would show zero unauthorized hits and meaningful reductions in recomputation/TTFT when authorized evidence repeats. A negative but useful result could identify a break-even sharing rate below which per-user isolation remains preferable.

Timeline and Workload Division

Week	Milestone	Owner
1-2	Literature review; vLLM setup; reproduce prefix-cache baselines	Both
3-4	ACL workload + middleware; implement scoped block hashing	M1 / M2
5	Security tests; one-boundary and per-user baselines	Both
6	Memory, overlap, concurrency experiments	M1
7	ACL-aware ordering + QA evaluation; optional heuristic reuse policy	M2
8	Repeat experiments; analysis, figures, report/presentation	Both

Go/no-go checkpoint: by the end of Week 4, two authorized users must share public/group blocks while an unauthorized user cannot hit those restricted blocks. If this fails, we switch to the simulator fallback.

Meeting Plan

Meeting frequency. We will hold at least one internal group meeting each week. **Task/progress tracking.** Milestones, tasks, bugs, and owners will be tracked through issues/projects in the shared GitHub repository. **Code/results sharing.** All code, configurations, experiment scripts, and a shared experiment log with run parameters/results will be maintained in the repository. **Instructor meetings.** We plan check-ins at major milestones—after baseline reproduction, prototype completion, and initial experiments—and will adjust scope early if needed.

Risk Mitigation

- **Risk - vLLM modification is harder than expected. Mitigation:** begin with block-aligned scope boundaries; if engine changes remain unstable, use a trace-driven cache simulator plus the same

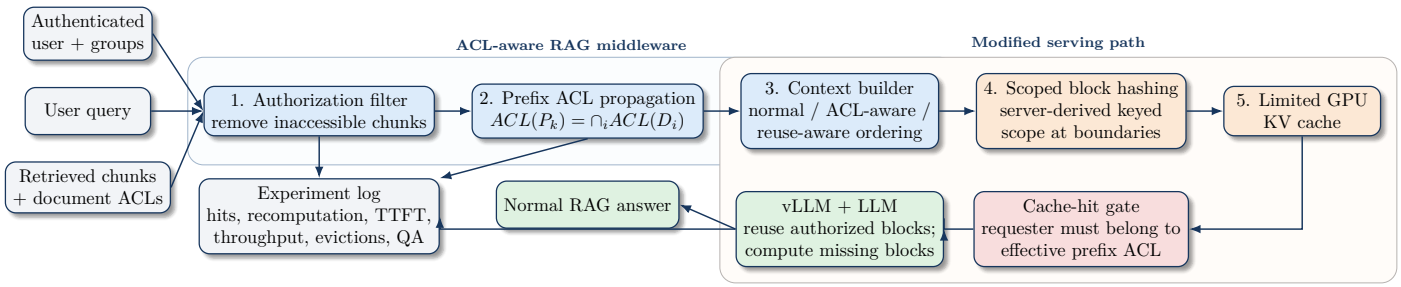


Figure 1: Proposed workflow. ACLs are used twice: first to construct a safely shareable prompt prefix, and then to enforce which cached blocks a requester may reuse. The user receives an ordinary RAG answer; the additional outputs are experimental measurements.

ACL-aware middleware to evaluate the policy.

- **Risk - evidence ordering hurts QA quality. Mitigation:** retain normal retrieval order as a baseline and limit how far high-ranked evidence may move; report both efficiency and QA quality.
- **Risk - little performance improvement. Mitigation:** vary document overlap, concurrency, ACL mix, and cache pressure to identify the break-even region. A negative result still answers RQ3.
- **Risk - timing measurements are noisy or GPU access is limited. Mitigation:** verify security primarily from deterministic cache-hit traces, repeat timing trials, and fall back to a smaller instruction model while preserving the same cache-policy experiments.

References

[1] vLLM Project. "Automatic Prefix Caching." vLLM documentation, 2026. https://docs.vllm.ai/en/v0.29.0/design/prefix_caching/

[2] vLLM Project. "Security: Prefix Cache Timing Side-Channel Mitigation (Cache Salting)." vLLM documentation, 2026. https://docs.vllm.ai/en/v0.29.0/usage/security/

[3] vLLM Project. "RFC: Cache Salting for Secure and Flexible Prefix Caching in vLLM." GitHub Issue #16016, 2025. https://github.com/vllm-project/vllm/issues/16016

[4] G. Wu et al. "CachePrune: Privacy-Aware and Fine-Grained KV Cache Sharing for Efficient LLM Inference." https://arxiv.org/abs/2605.23640

[5] T. C. Addagada. "Governing the KV Cache: Preventing Timing Side-Channel Leakage in Multi-Tenant LLM Inference." https://arxiv.org/html/2608.09225

[6] K. Tan, R. Gu, and M. Li. "CacheWeaver: Cache-Aware Evidence Ordering for Efficient Grounded RAG Inference." https://arxiv.org/abs/2606.19667.

[7] vLLM Project. "Cache Configuration." vLLM documentation, 2026. https://docs.vllm.ai/en/v0.29.0/api/vllm/config/cache/

[8] P. G. Pennas, K. Papaioannou, M. Guarnieri, and T. D. Doudali. "PrefixWall: Mitigating Prefix Caching Side Channels in Shared LLM Systems." https://arxiv.org/abs/2603.10726

[9] vLLM Project. "vLLM DoS via unbounded cache_salt length: multi-hundred-MB salt stalls the single EngineCore scheduler thread (CPU exhaustion)." https://github.com/vllm-project/vllm/security/advisories/GHSA-wpww-v874-ph2p